Inverter Árvore Binária (LeetCode 226)

Fernando De Riggi
Filipe Braga
Murilo Corrêa
Sabrina Andrade

INTRODUÇÃO

LeetCode 226 — Invert Binary Tree



Problema

Inverter uma árvore binária, trocando os filhos esquerdo e direito de cada nó.
O resultado é a versão espelhada da árvore original



Abordagem

Utilizar recursão para percorrer a árvore e trocar os filhos de cada nó.
O processo continua até chegar aos nós folha.



Relevância

Trabalha conceitos fundamentais como árvores binárias e recursão.
Também é um problema clássico em entrevistas técnicas, conhecido principalmente pelo Google.

CONTEXTO HISTÓRICO

O caso do Homebrew e do Google

Em uma entrevista técnica do Google, Max Howell, criador do Homebrew, recebeu um problema envolvendo a inversão de uma árvore binária.
Apesar de sua experiência como desenvolvedor, ele não conseguiu resolver o problema durante a entrevista.

“Can’t invert a binary tree on a whiteboard? Then we don’t want you.”

Curiosidade

- O problema se tornou famoso na comunidade de programação e posteriormente foi catalogado no LeetCode como:
- #226 — Invert Binary Tree
- O episódio aconteceu antes da criação do LeetCode; portanto, não foi literalmente uma questão “do LeetCode 226” na entrevista.



TESTE DAS "BIG TECHS"

Quais empresas pedem e o que cobram?

Google Meta amazon Microsoft

Ponteiros e Alocação de Memória

O domínio absoluto sobre Ponteiros e Alocação de Memória.

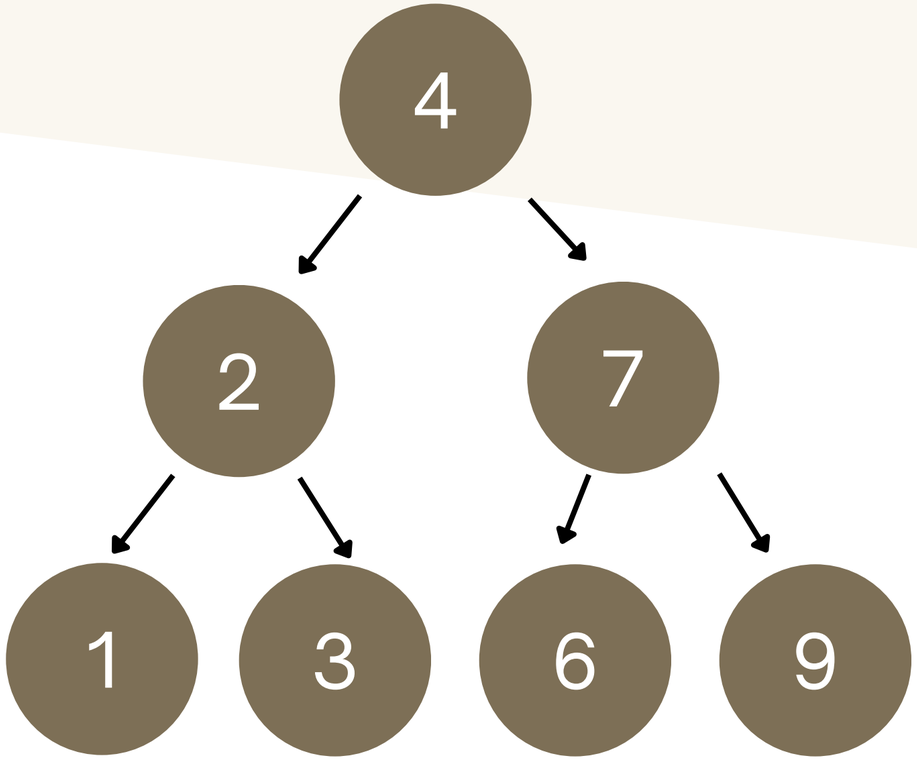
Recursão e Call Stack

A compreensão do candidato sobre Recursão e o funcionamento da Call Stack (Pilha de chamadas).

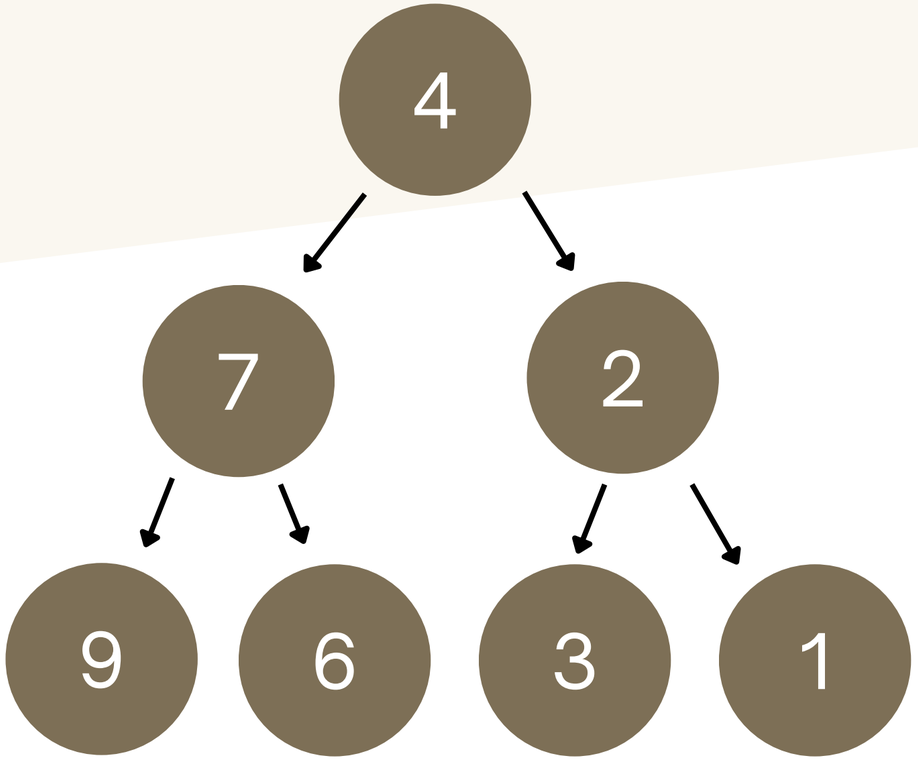
Manipulação de Referências

A habilidade de manipular referências sem causar erros fatais (como Segmentation Fault).

Árvore Original

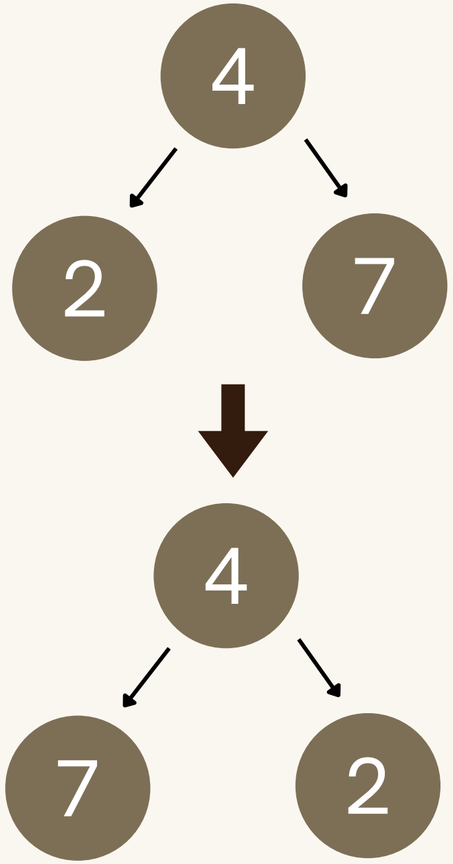


Árvore Invertida



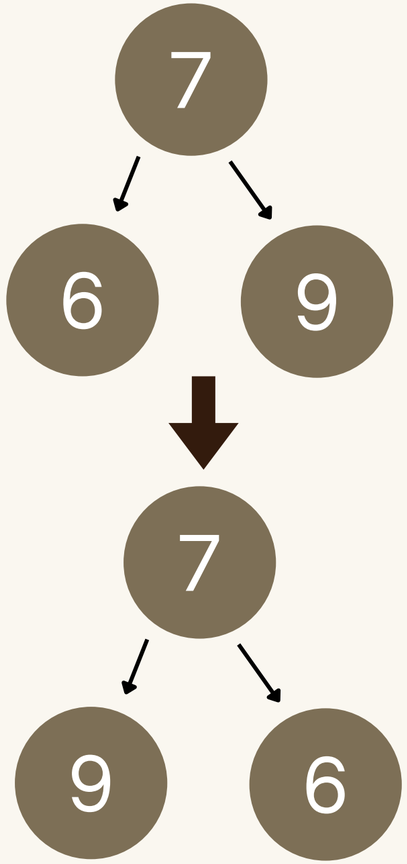
ESTRATÉGIA

Trocar os Filhos



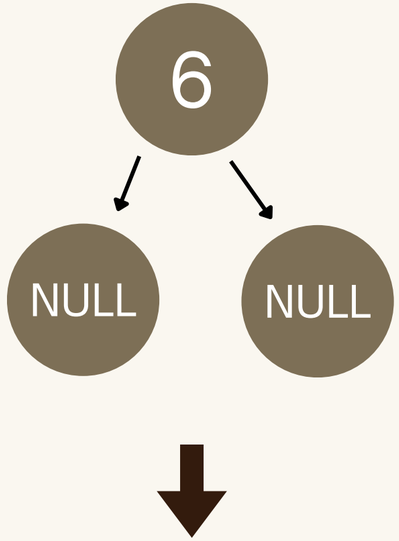
Trocar os filhos esquerdo e direito.

Aplicar a mesma ideia aos filhos



Repetir o processo para cada filho.

Parar



Fim

Se o nó for NULL, parar.

Código inteiro

```c
struct TreeNode* invertTree(struct TreeNode* root) {
    if (root == NULL)
        return NULL;

    struct TreeNode* temp = root->left;
    root->left = root->right;
    root->right = temp;

    invertTree(root->left);
    invertTree(root->right);

    return root
}
```

Caso-base

```c
if (root == NULL)
    return NULL;
```

Encerra a recursão.

Troca dos filhos

```c
struct TreeNode* temp = root->left;
root->left = root->right;
root->right = temp;
```

Espelha o nó atual.

Espelha o nó atual

```c
invertTree(root->left);
invertTree(root->right);
```

Repete o processo nas subárvores.

Complexidade temporal - O(n)

```c
struct TreeNode* invertTree(struct TreeNode* root) {
    if (root == NULL)
        return NULL;

    struct TreeNode* temp = root->left;
    root->left = root->right;
    root->right = temp;

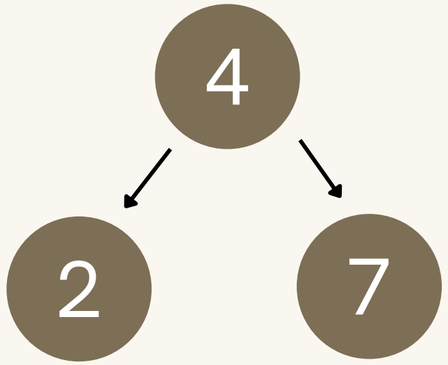
    invertTree(root->left);
    invertTree(root->right);

    return root
}
```

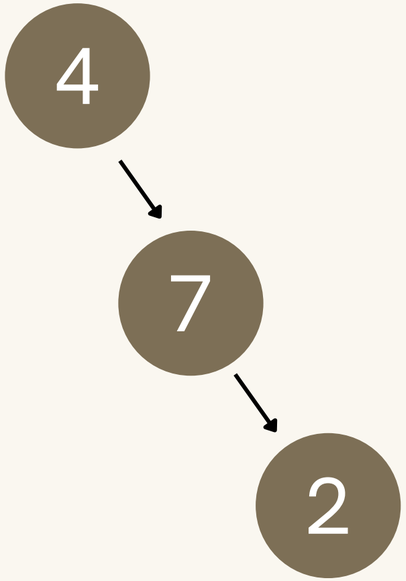
1 chamada por nó → crescimento linear

Complexidade espacial - O(h)

Balanceada - O(log N)



Desbalanceada - O(N)



Espaço extra = O(H)

Inverter Árvore Binária (LeetCode 226)

Fernando De Riggi
Filipe Braga
Murilo Corrêa
Sabrina Andrade